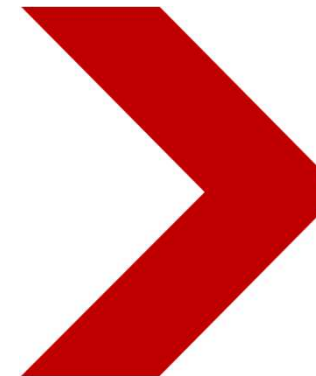


Java程序设计基础

Java Programming Fundamentals

Wrapper Classes



包装类

- **Java中的数据类型**
 - 基本数据类型 byte, short, int, long, float, double, char, boolean
 - 引用数据类型
- **基本数据类型**
 - 无面向对象特性, 无调用方法
- **包装类--把基本类型“包装”成对象, 赋予它对象的功能**
 - 属于java.lang包, 使用时无需import包。

基本类型与包装类对应表

基本数据类型	包装类
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

相关术语

- 基本数据类型: Primitive Data Type
- 引用数据类型: Reference Data Type
- 装箱: Boxing
- 拆箱: Unboxing
- 自动装箱: Auto Boxing
- 自动拆箱: Auto Unboxing

装箱：基本类型 → 包装类对象

// 手动装箱：int → Integer

```
int a = 10;
```

```
Integer num = new Integer(a);           // 构造方法（不推荐）
```

```
Integer num2 = Integer.valueOf(a);      // 静态方法（推荐）
```

// 手动拆箱：Integer → int

```
int b = num.intValue();
```

// 其他包装类同理

```
double c = 3.14;
```

```
Double d = Double.valueOf(c);
```

```
double e = d.doubleValue();
```

自动装箱/自动拆箱 (JDK1.5+)

// 自动装箱：直接赋值

```
Integer num = 10;  
Boolean flag = true;  
Character ch = 'A';
```

// 自动拆箱：包装类直接赋值给基本类型

```
int a = num;  
double b = Double.valueOf(3.14);
```

// 运算时也会自动拆箱 Integer x = 5;

```
Integer y = 3;  
int sum = x + y; // 先拆箱，再运算
```

基本类型 ↔ 字符串 转换

```
int a = 100;
// 方法1: 包装类toString
String str1 = Integer.toString(a);
// 方法2: 字符串拼接
String str2 = a + "";
// 方法3: 直接调用对象toString
Integer num = a;
String str3 = num.toString();
```

```
String str = "123";
// 字符串转int
int num = Integer.parseInt(str);
// 字符串转Integer对象
Integer numObj = Integer.valueOf(str);
// 其他类型转换
String doubleStr = "3.14";
double d = Double.parseDouble(doubleStr);
String boolStr = "true";
boolean b = Boolean.parseBoolean(boolStr);
```

包装类的实用场景

- 用于集合类存储数据

- Java集合 (如List、Set、Map) 只能存储引用类型, 不能直接存放基本数据类型, 必须使用包装类。

```
import java.util.ArrayList;
import java.util.List;

public class WrapperDemo {
    public static void main(String[] args) {
        // 存储整数, 必须用Integer, 不能用int
        List<Integer> list = new ArrayList<>();
        // 自动装箱
        list.add(10);
        list.add(20);
        list.add(30);

        // 取出数据, 自动拆箱
        int first = list.get(0);
        System.out.println("集合第一个元素: " + first);
    }
}
```


包装类的实用场景

- 表示可空的数值类型

- 基本数据类型有默认值，无法区分

“未赋值”和“值为0”，包装类可以赋值为null，适合表示缺失值、空数据。

```
public class Student {  
    // 基本类型：score未赋值时默认为0，无法判断是缺考还是0分  
    // int score;  
  
    // 包装类：可赋值为null，表示成绩未录入  
    Integer score;  
  
    public static void main(String[] args) {  
        Student stu = new Student();  
        stu.score = null; // 表示成绩缺失  
        if (stu.score == null) {  
            System.out.println("该学生成绩未录入");  
        }  
    }  
}
```

实用场景-泛型编程

泛型参数必须是引用类型，不支持基本数据类型，定义泛型类、泛型方法时必须使用包装类。

```
// 泛型类，存放数值数据
public class DataBox<T> {
    private T data;

    public T getData() { return data; }

    public void setData(T data) {this.data = data;}

    public static void main(String[] args) {
        // 使用Integer作为泛型参数
        DataBox<Integer> intBox = new DataBox<>();
        intBox.setData(666);

        DataBox<Double> doubleBox = new DataBox<>();
        doubleBox.setData(99.99);
    }
}
```



实用场景-数据类型转换与工具方法调用

包装类提供大量实用工具方法，完成进制转换、数值判断、字符校验等操作，基本类型无法直接调用。

```
public class WrapperUtil {  
    public static void main(String[] args) {  
        // 十进制转二进制  
        String binary = Integer.toBinaryString(10);  
        System.out.println("10的二进制: " + binary);  
  
        // 判断字符是否为数字、字母  
        boolean isDigit = Character.isDigit('8');  
        boolean isLetter = Character.isLetter('A');  
  
        // 字符串转数值并计算  
        String num1 = "100";  
        String num2 = "200";  
        int sum = Integer.parseInt(num1) + Integer.parseInt(num2);  
        System.out.println("求和结果: " + sum);  
    }  
}
```



实用场景-方法参数与返回值兼容

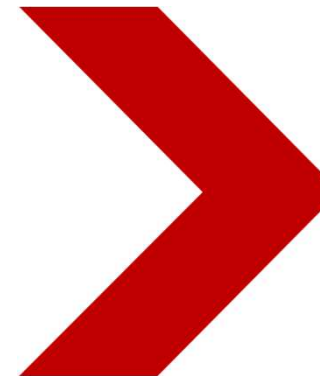
编写通用方法时，
使用包装类作为
参数或返回值，
适配更多场景，
支持null返回，
提升方法灵活性。

```
public class CalcDemo {  
    // 返回值为Integer, 可返回null表示计算失败  
    public static Integer divide(int a, int b) {  
        if (b == 0) {  
            return null; // 除零异常, 返回null  
        }  
        return a / b;  
    }  
  
    public static void main(String[] args) {  
        Integer result = divide(10, 0);  
        if (result == null) {  
            System.out.println("计算失败, 除数不能为0");  
        } else {  
            System.out.println("计算结果: " + result);  
        }  
    }  
}
```

Java程序设计基础

Java Programming Fundamentals

Inner Class



Inner Class – 内部类

- 在类内部也可以定义另一个类。
 - 如果在类Outer的内部再定义一个类Inner，此时类Inner就称为内部类，而类Outer则称为外部类。
- 内部类可声明成public或private。
 - 当内部类声明成public或private时，对其访问的限制与成员变量和成员方法完全相同。

- 内部类的定义格式

```
标识符 class 外部类的名称{  
    // 外部类的成员  
    标识符 class 内部类的名称{  
        // 内部类的成员  
    }  
}
```

Inner Class – 内部类

外部访问内部类格式:

外部类名称.内部类名称 内部类对象名=外部类实例.new 内部类构造方法()

```
class Outer {  
    private static String info = "hello world!!!";  
    class Inner { // 内部类  
        public void print() {  
            System.out.println(info);  
        }  
    };  
};  
  
public class InnerClassDemo03 {  
    public static void main(String args[]) {  
        Outer out=new Outer();  
        Outer.Inner in=out.new Inner()  
        in.print();  
    }  
}
```

Inner Class – 内部类

外部访问使用static定义的内部类:

外部类名称.内部类名称 对象名称 = new 外部类名称.内部类构造方法()

```
class Outer {  
    private static String info = "hello world!!!";  
    static class Inner { // 使用static定义内部类  
        public void print() {  
            System.out.println(info);  
        }  
    };  
};  
  
public class InnerClassDemo03 {  
    public static void main(String args[]) {  
        new Outer.Inner().print(); // 访问内部类  
    }  
}
```


Inner Class – 内部类

在方法中定义内部类，使外部调用

```
class Outer{           // 定义外部类
    private String info = "hello world" ; // 定义外部类的私有属性
    public void fun(final int temp){       // 定义外部类的方法
        class Inner{                      // 在方法中定义的内部类
            public void print(){           // 定义内部类的方法
                System.out.println("类中的属性: " + info) ; // 直接访问外部类的私有属性
                System.out.println("方法中的参数: " + temp) ;
            }
        };
        new Inner().print() ;              // 通过内部类的实例化对象调用方法
    }
};

public class InnerClassDemo05{
    public static void main(String args[]){
        new Outer().fun(30) ;              // 调用外部类的方法
    }
};
```

Inner Class – 内部类

- 成员内部类
- 静态内部类
- 局部内部类
- 匿名内部类

```
1  public class Outer {  
2        
3      public class Inner{  
4            
5      }  
6  }  
7    
8  |
```

成员内部类

- 内部类 Inner 可直接访问外部类 Outer 的私有变量 num 和方法

```
// 外部类
public class Outer {
    private int num = 10; // 外部类的私有变量

    // 成员内部类（非静态）
    class Inner {
        public void show() {
            System.out.println("访问外部类的私有变量 num = " + num);
            // 可直接调用外部类方法，例如：outerMethod();
        }
    }
}
```

```
// 外部类方法中实例化内部类
public void createInner() {
    Inner inner = new Inner();
    inner.show();
}

public static void main(String[] args) {
    Outer outer = new Outer();
    outer.createInner(); // 方式1：通过外部类方法调用

    // 方式2：直接实例化内部类
    Outer.Inner inner = outer.new Inner();
    inner.show();
}
```

静态内部类

- 内部类是外部类的静态成员

```
class DatabaseHelper {  
    static class QueryBuilder { // 静态内部类  
        static void buildSQL(String table) {  
            System.out.println("SELECT * FROM " + table);  
        }  
    }  
}  
  
// 调用方式: DatabaseHelper.QueryBuilder.buildSQL("users");
```

局部内部类

- 内部类被定义在类的某个方法中（局部，非成员）

```
public void processData() {  
    class DataValidator { // 仅在方法内有效  
        void check(int value) {  
            if (value < 0) throw new IllegalArgumentException();  
        }  
    }  
    new DataValidator().check(100);  
}
```

匿名内部类

- 没有名字的内部类
 - 匿名内部类只能使用一次，它通常用来简化代码编写
 - 使用匿名内部类的前提条件：必须继承一个父类或实现一个接口
 - 当一个抽象类或接口的子类只需要使用一次的时候就可以使用匿名内部类的定义格式。

```
button.addActionListener(new ActionListener() { // 匿名内部类实现接口
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("按钮被点击");
    }
});
```

匿名内部类增加代码的简洁性

```
abstract class Person {
    public abstract void eat();
}
class Child extends Person {
    public void eat() {
        System.out.println("eat something");
    }
}
public class Demo {
    public static void main(String[] args) {
        Person p = new Child();
        p.eat();
    }
}
```

```
abstract class Person {
    public abstract void eat();
}
public class Demo {
    public static void main(String[] args) {
        Person p = new Person() {
            @Override
            public void eat() {
                System.out.println("eat something");
            }
        };
        p.eat();
    }
}
```

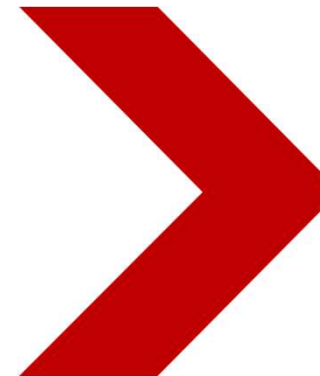
在接口interface上使用匿名内部类

```
interface Person {  
    public abstract void eat();  
}  
public class Demo {  
    public static void main(String[] args) {  
        Person p = new Person() {  
            @Override  
            public void eat() {  
                System.out.println("eat something");  
            }  
        };  
        p.eat();  
    }  
}
```

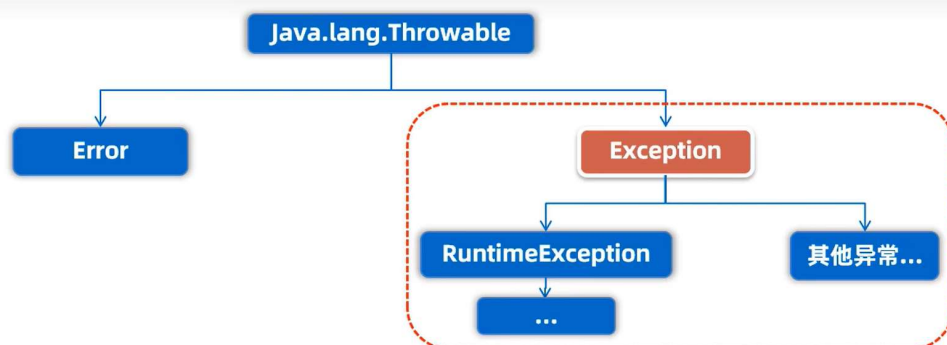

Java程序设计基础

Java Programming Fundamentals

Exception



Exception



Error: 代表的系统级别错误(属于严重问题), 也就是说系统一旦出现问题, sun公司会把这些问题封装成Error对象给出来, 说白了, Error是给sun公司自己用的, 不是给我们程序员用的, 因此我们开发人员不用管它。

Exception: 叫异常, 它代表的才是我们程序可能出现的问题, 所以, 我们程序员通常会用Exception以及它的孩子来封装程序出现的问题。

- **运行时异常:** RuntimeException及其子类, 编译阶段不会出现错误提醒, 运行时出现的异常 (如: 数组索引越界异常)
- **编译时异常:** 编译阶段就会出现错误提醒的。 (如: 日期解析异常)

Java 异常类层次图

Throwable (所有异常/错误的根类)

```

|—— **Error (严重系统错误, 不可恢复) **
|   |—— OutOfMemoryError (内存耗尽)
|   |—— StackOverflowError (栈溢出)
|   |—— NoClassDefFoundError (类定义缺失)
|
|—— **Exception (程序可处理的异常) **
|   |—— **RuntimeException (运行时异常, 未受检异常) **
|       |—— NullPointerException (空指针)
|       |—— ArrayIndexOutOfBoundsException (数组越界)
|       |—— ClassCastException (类型转换错误)
|       |—— IllegalArgumentException (非法参数)
|
|   |—— **Checked Exception (受检异常, 需显式处理) **
|       |—— IOException (输入输出异常)
|           |—— FileNotFoundException (文件未找到)
|       |—— SQLException (数据库操作异常)
|       |—— InterruptedException (线程中断)
    
```

运行时异常 (RuntimeException 及其子类)

异常类	触发场景
NullPointerException	空引用调用方法或属性 (如 <code>String s = null; s.length();</code>) 。
ArrayIndexOutOfBoundsException	数组索引越界 (如访问 <code>arr[-1]</code> 或 <code>arr[arr.length]</code>) 。
ClassCastException	强制类型转换失败 (如将 <code>Object obj = "abc"; Integer num = (Integer)obj;</code>) 。
ArithmeticException	算术运算错误 (如整数除以零 <code>10 / 0</code>) 。
IllegalArgumentException	方法参数非法 (如向要求正数的参数传递负数) 。
NumberFormatException	字符串转数值失败 (如 <code>Integer.parseInt("abc")</code>) 。
IllegalStateException	对象状态不满足操作条件 (如未初始化时调用方法) 。

受检异常 (Checked Exception)

- 需显式捕获 (try-catch) 或声明抛出 (throws) , 通常由外部因素引发。

异常类	触发场景
IOException	I/O 操作失败 (如文件读写、网络中断) 。
FileNotFoundException	访问不存在的文件或路径。
SQLException	数据库操作错误 (如 SQL 语法错误、连接断开) 。
ClassNotFoundException	加载不存在的类 (如通过类名反射时未找到类) 。
InterruptedException	线程被中断 (如睡眠中的线程被 interrupt() 唤醒) 。

错误（Error 及其子类）

- 严重系统级错误，通常无法通过代码修复，需优化环境或资源。

错误类	触发场景
OutOfMemoryError	JVM 堆内存耗尽（如创建超大对象或内存泄漏）。
StackOverflowError	方法递归调用过深（如无限递归导致栈溢出）。
NoClassDefFoundError	JVM 找不到类的定义（如编译后类文件被删除）。

try-catch-finally

```
try{  
    statements;  
}catch (ExceptionClass1 e){  
    ...  
}catch (ExceptionClass2 e){  
    ...  
}finally{  
    ....  
}
```

- **Method of Exception Object**
 - **public String getMessage();**
 - **public void printStackTrace();**
 - **public String toString();**

User define exception class

- **derived from Exception**
 - 一般从Exception类派生
- **throws**
 - 函数首部末尾，声明可能抛出的异常类型
 - 可声明多个异常类型（根据需要抛出，可以不实际抛出）
- **throw**
 - 在函数体内抛出具体的异常对象，每次只能抛出一个异常对象
 - 效果：中断当前方法，触发异常处理流程

Assert

- **assert boolExp;**
 - 当boolExp为true时, 继续执行,
 - 否则, 程序立刻停止执行
- **assert boolExp:messageException;**
 - 当boolExp为true时, 继续执行
 - 否则, 立即停止执行并输出messageException的值